

Chapter 13

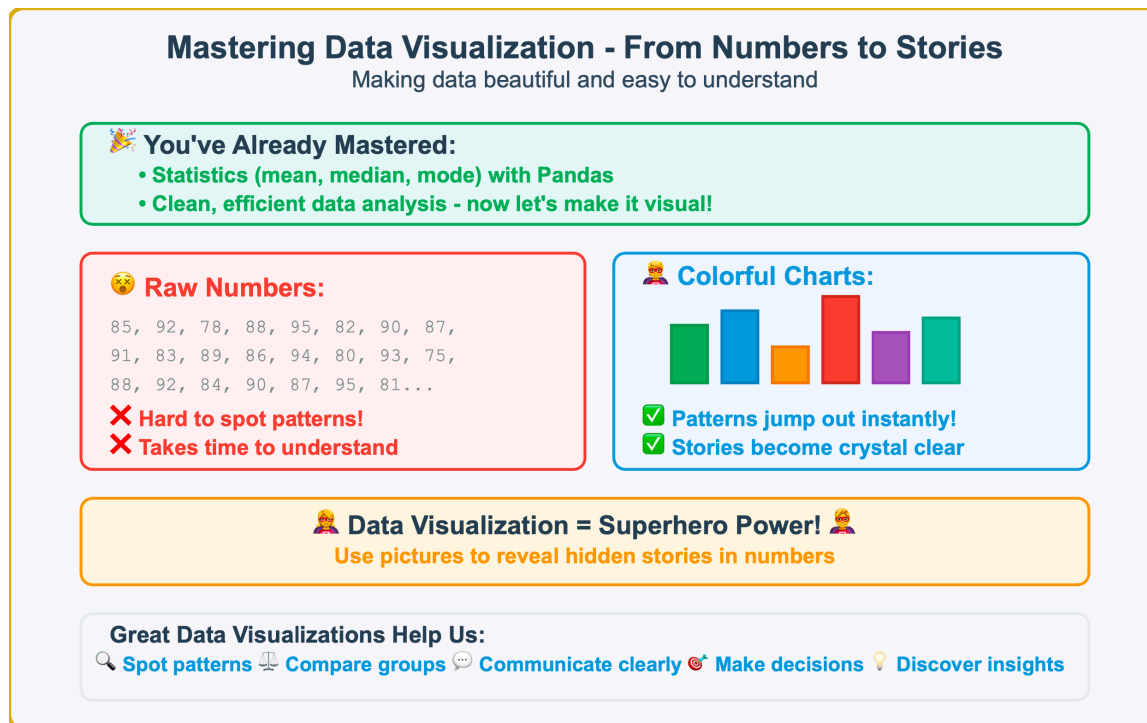
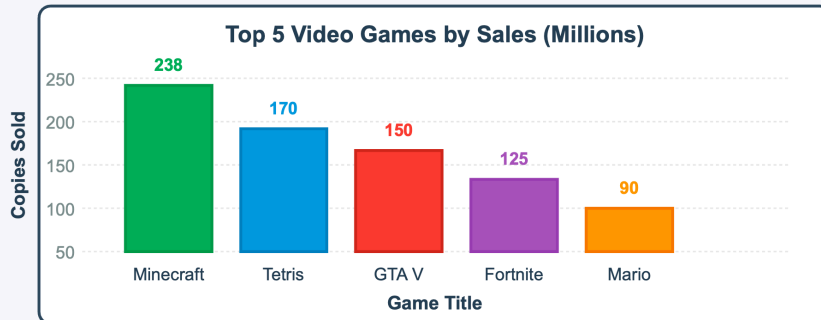


Figure 13.1 — Mastering Data Visualization — From Numbers to Stories. This introduction demonstrates how data visualization transforms confusing raw numbers into clear, colorful charts that reveal patterns instantly and help communicate findings effectively.

Understanding Bar Charts - From Turtle to Matplotlib

Perfect for comparing categories - now super simple with professional tools!



Turtle Way:

- Manual positioning
- Draw each bar separately
- Calculate coordinates
- **30+ lines of code**
- ✗ Time consuming

Matplotlib Magic:

- Automatic everything!
- Professional styling
- Built-in features
- **Just 1 command!**
- ✓ Fast beautiful

The Magic Code:

```
plt.bar(games, sales, color=['green', 'blue', 'red', 'purple', 'orange'])
```

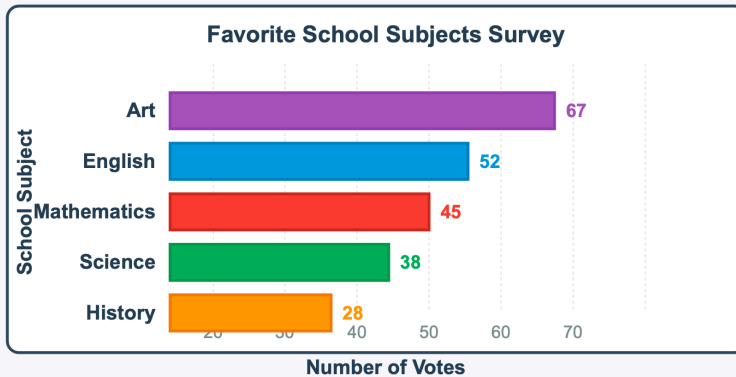
🌀 One line creates a professional chart with:

- Automatic scaling and positioning
- Custom colors
- Professional appearance
- **No manual calculations needed - matplotlib handles everything!**

Figure 13.2 — Understanding Bar Charts — From Turtle to Matplotlib. This comparison shows how matplotlib transforms the complex manual process of creating bar charts into simple, professional visualizations with just one command, demonstrating the power of specialized visualization tools.

Horizontal Bar Charts - Better for Long Category Names

Perfect for readable labels - just one small code change makes a big difference!



When to Use Horizontal:

- Long category names
- Better text readability
- More space for labels
- **Easier comparison**

Simple Code Change:

```
plt.bar() → plt.barh()
```

```
grid(axis='x')
```

That's it! Super easy!

Key Insight: Readability Matters!

Notice how easy it is to read "Mathematics" and "English" when they're horizontal compared to cramped vertical text. This makes your charts much more professional!

👉 **Rule of thumb: If category names are longer than 6-8 characters, go horizontal!**

✅ Your readers will thank you for the improved readability

Figure 13.3 — Horizontal Bar Charts — Better for Long Category Names. This example shows how horizontal bars solve the readability problem of long category labels by providing more space for text, with just a simple code change from `plt.bar()` to `plt.barh()`.

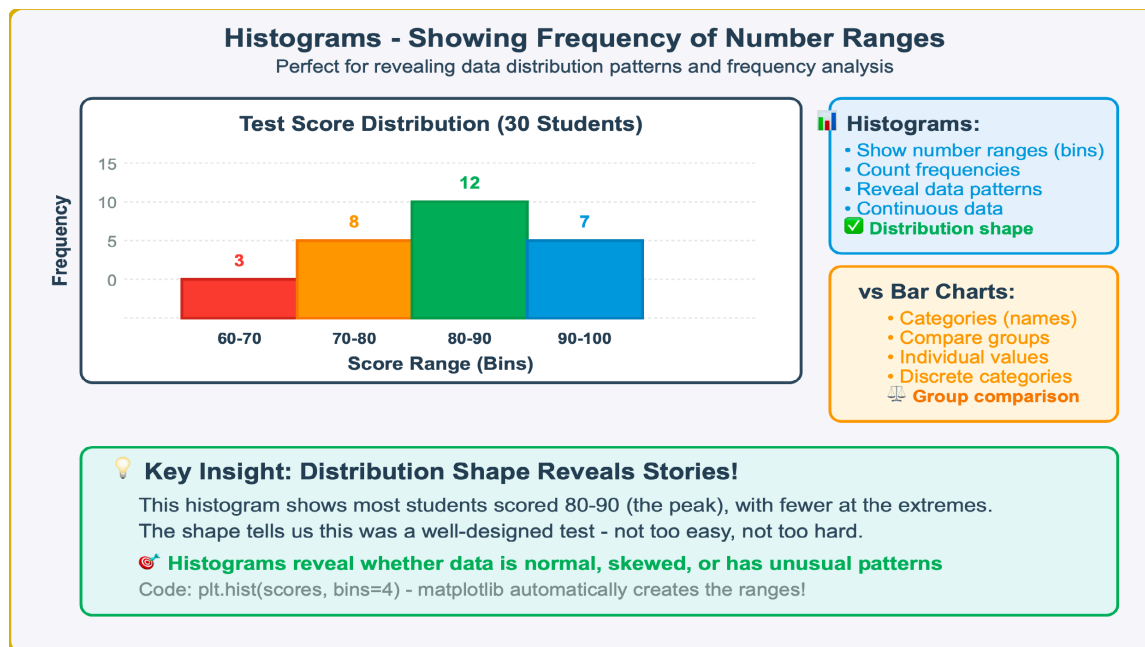


Figure 13.4 — Histograms — Showing Frequency of Number Ranges. This example demonstrates how histograms reveal data distribution patterns by grouping continuous values into ranges (bins), showing how many data points fall within each range rather than comparing distinct categories.

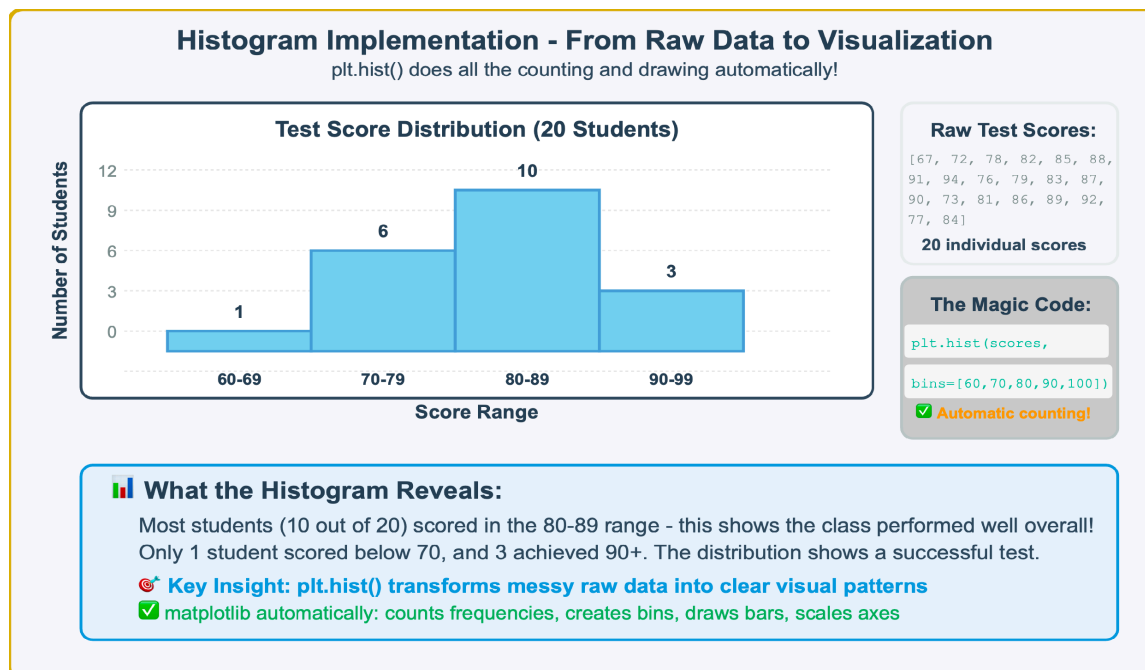


Figure 13.5 — Histogram Implementation — From Raw Data to Visualization. This example demonstrates how `plt.hist()` automatically counts frequencies and creates bins from raw test score data, revealing that most students scored in the 80—89 range with matplotlib handling all the complex calculations.

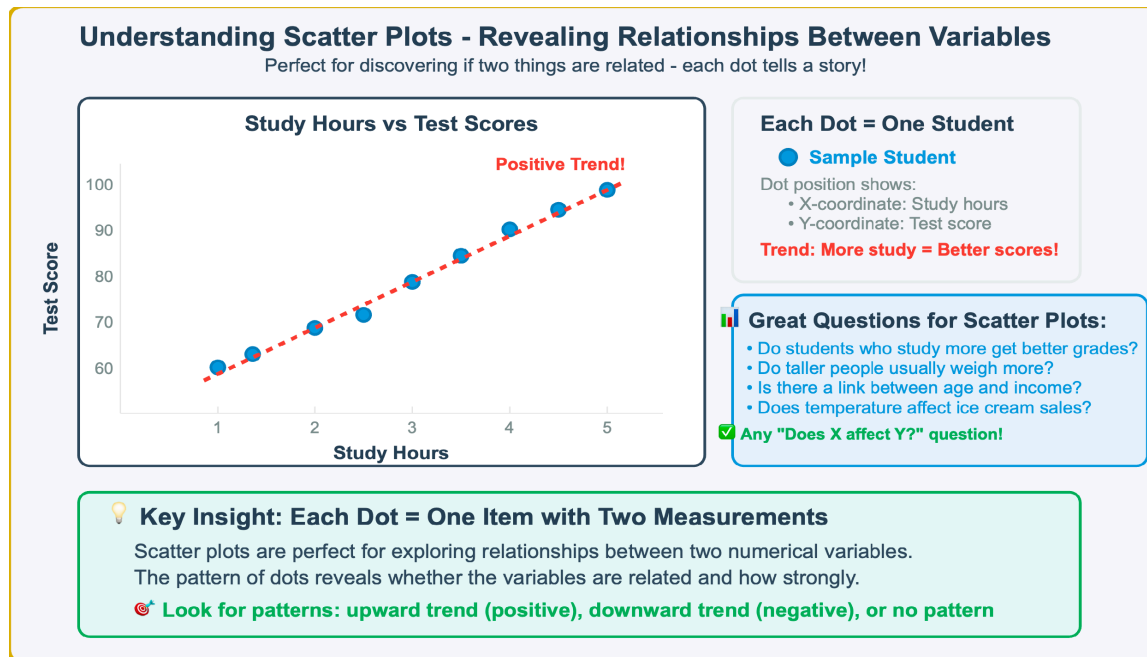


Figure 13.6 — Understanding Scatter Plots — Revealing Relationships Between Variables. This example demonstrates how scatter plots show the relationship between two variables (study hours and test scores), with each dot representing one student and the trend line revealing that more study time generally leads to better scores.

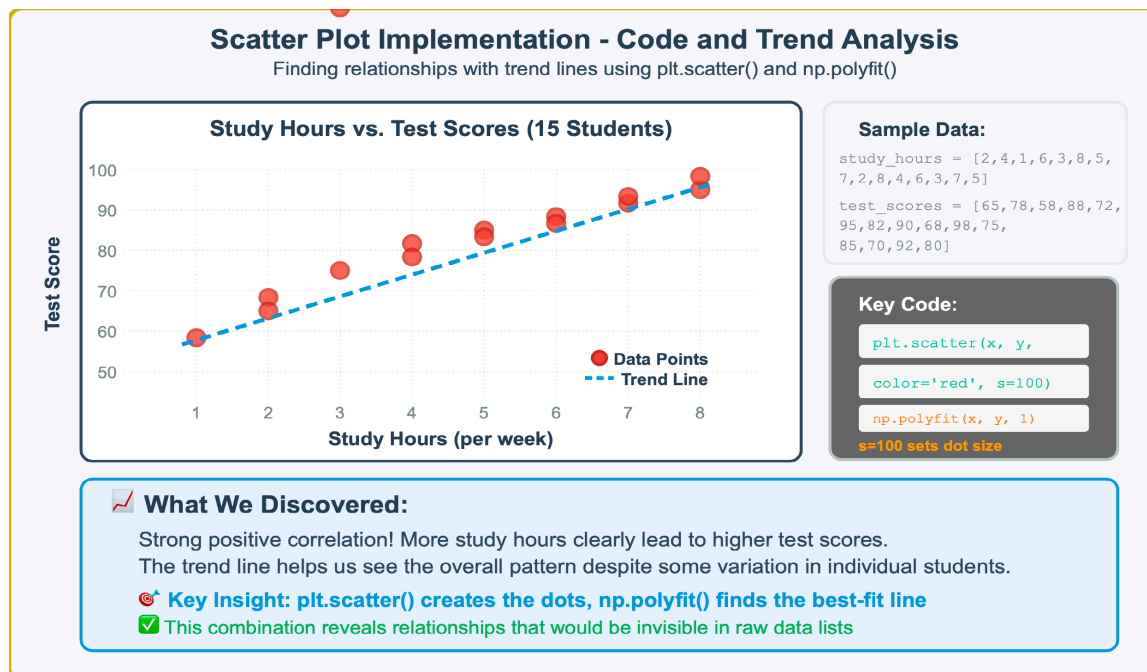


Figure 13.7 — Scatter Plot Implementation — Code and Trend Analysis. This example demonstrates how to create scatter plots with `plt.scatter()` and add trend lines using `np.polyfit()`, revealing a strong positive correlation between study hours and test scores with practical code implementation.

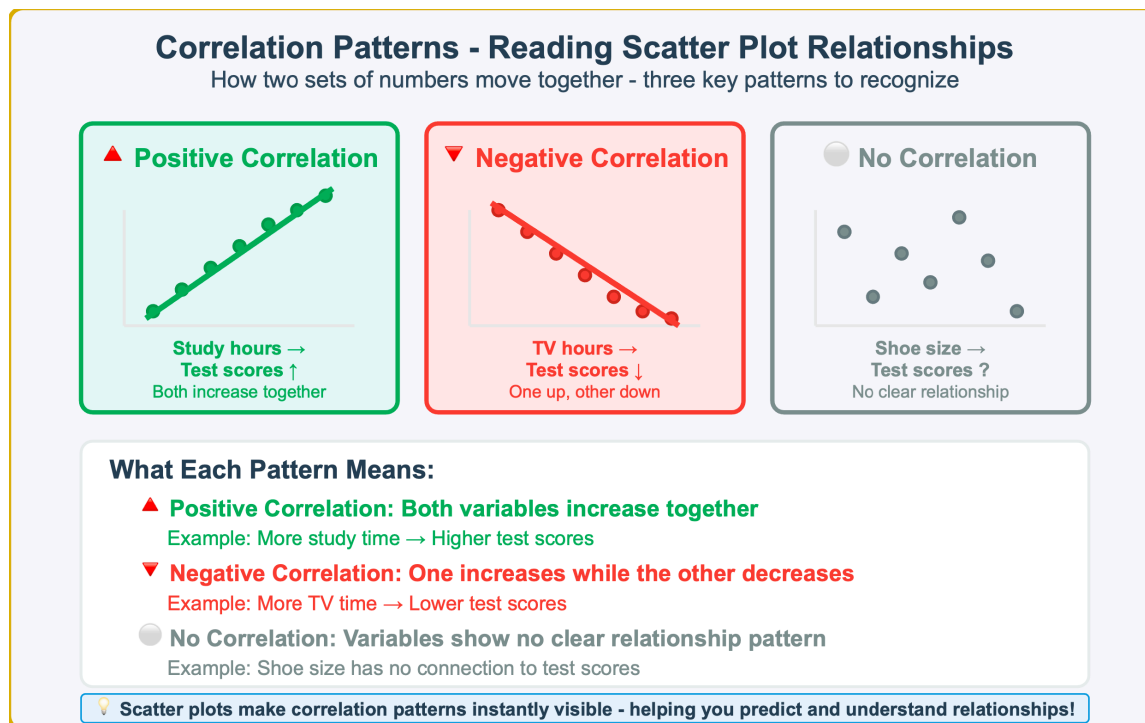


Figure 13.8 — Correlation Patterns — Reading Scatter Plot Relationships. This comparison shows the three main correlation types: positive (both variables increase together), negative (one increases while the other decreases), and no correlation (no clear relationship pattern).

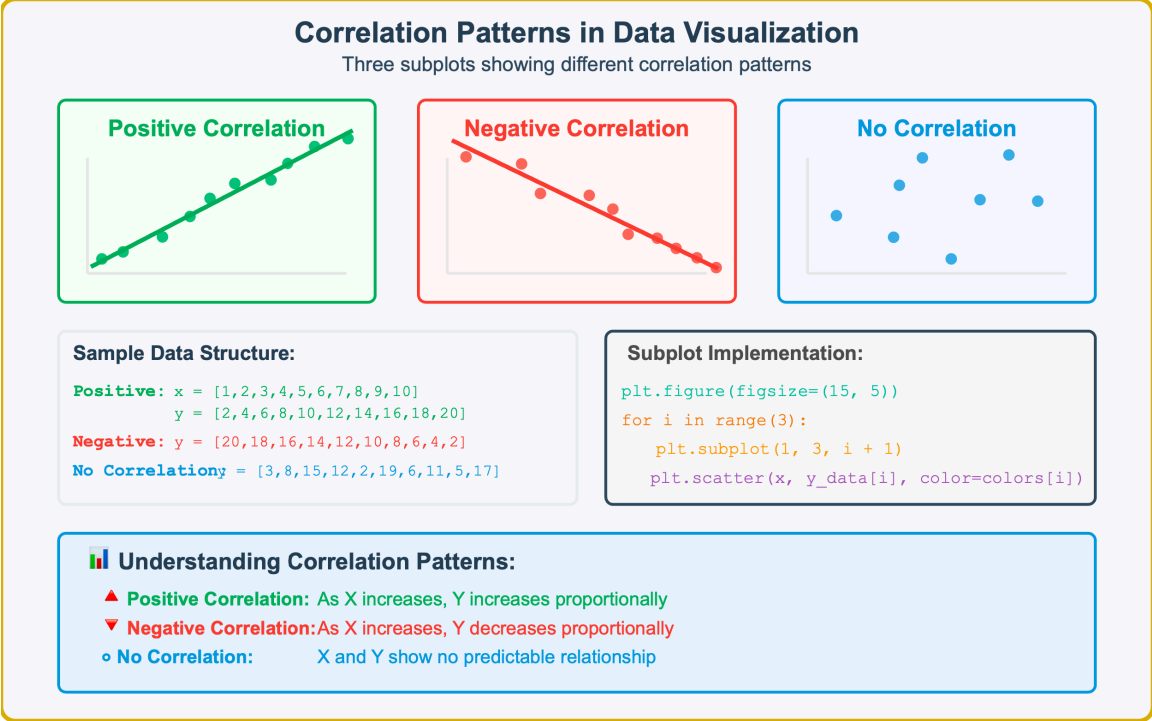


Figure 13.9 — Correlation Patterns in Data Visualization. Three subplots demonstrate positive correlation (upward trend), negative correlation (downward trend), and no correlation (random scatter). The subplot implementation shows how to create multiple visualizations for comparative analysis.

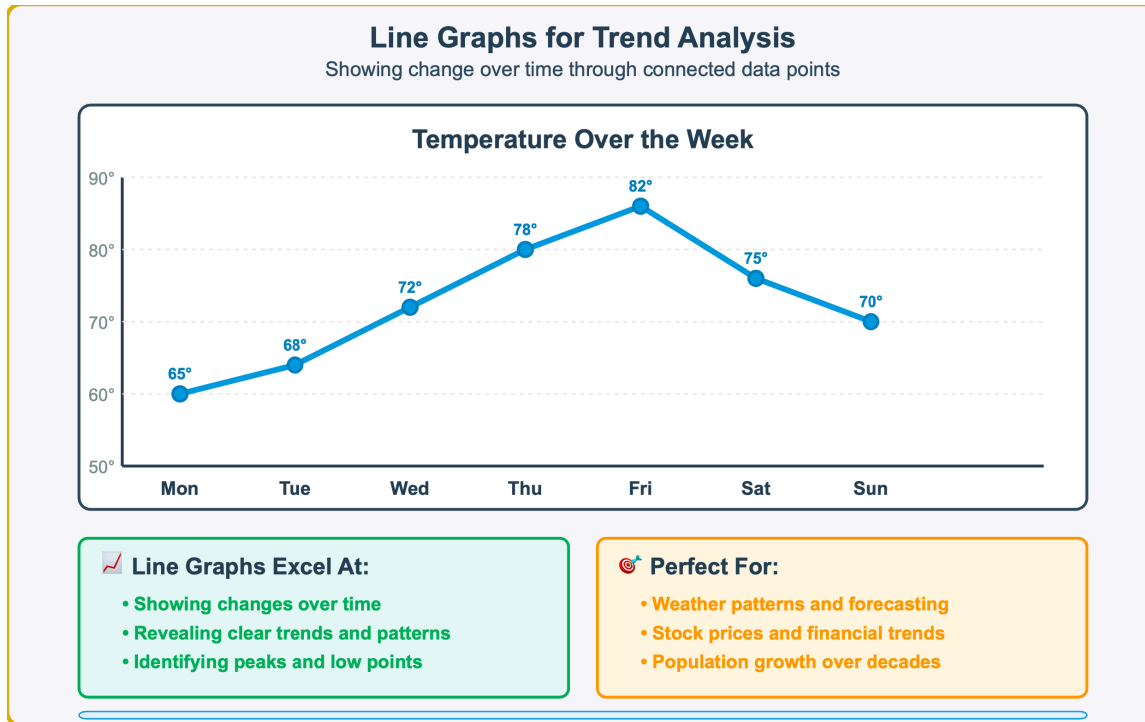


Figure 13.10 — Line Graphs for Trend Analysis. A weekly temperature chart demonstrates how line graphs effectively show changes over time, revealing patterns, peaks, and trends through connected data points.

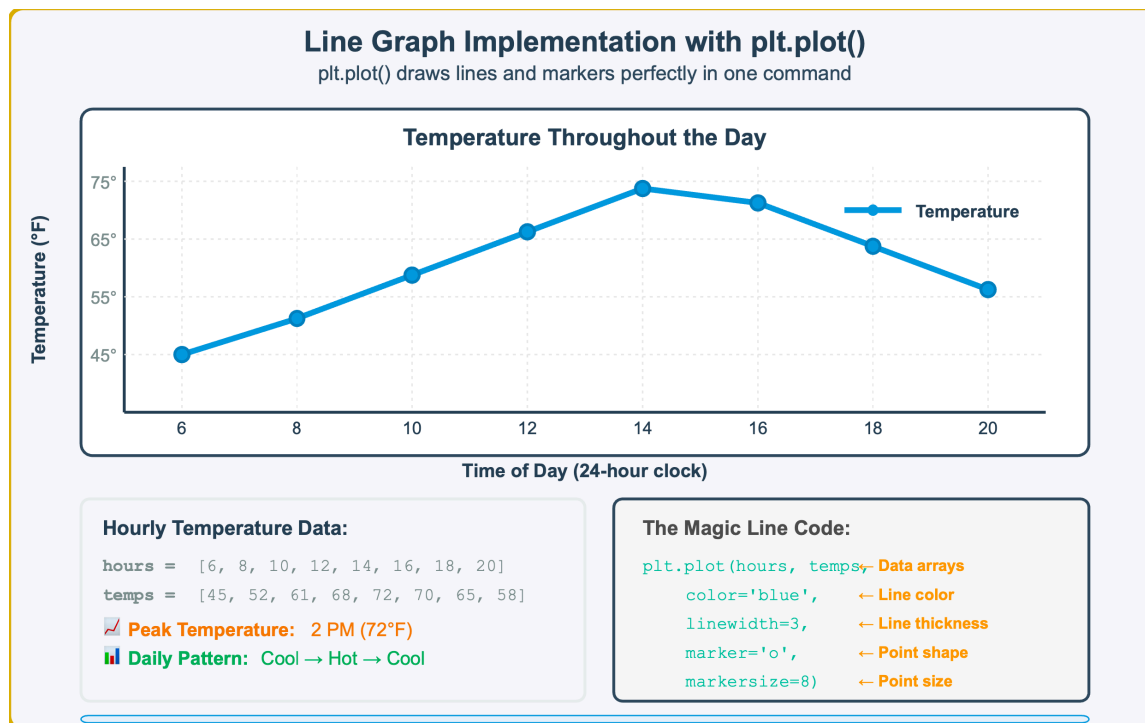


Figure 13.11 — Line Graph Implementation with plt.plot(). This hourly temperature chart demonstrates how plt.plot() creates both the connecting line and data point markers in a single command, revealing daily temperature patterns from morning cooldown to afternoon peak.

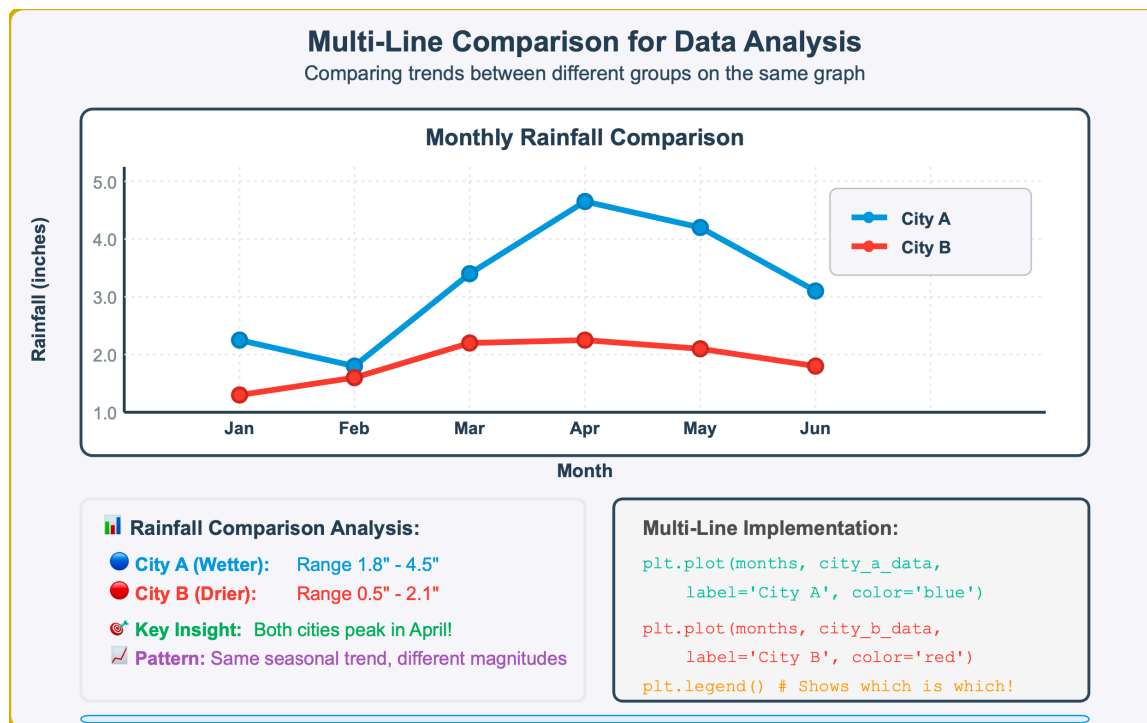


Figure 13.12 — Multi—Line Comparison for Data Analysis. Monthly rainfall data for two cities demonstrates how multiple lines on the same graph reveal both differences in magnitude and similarities in seasonal patterns.

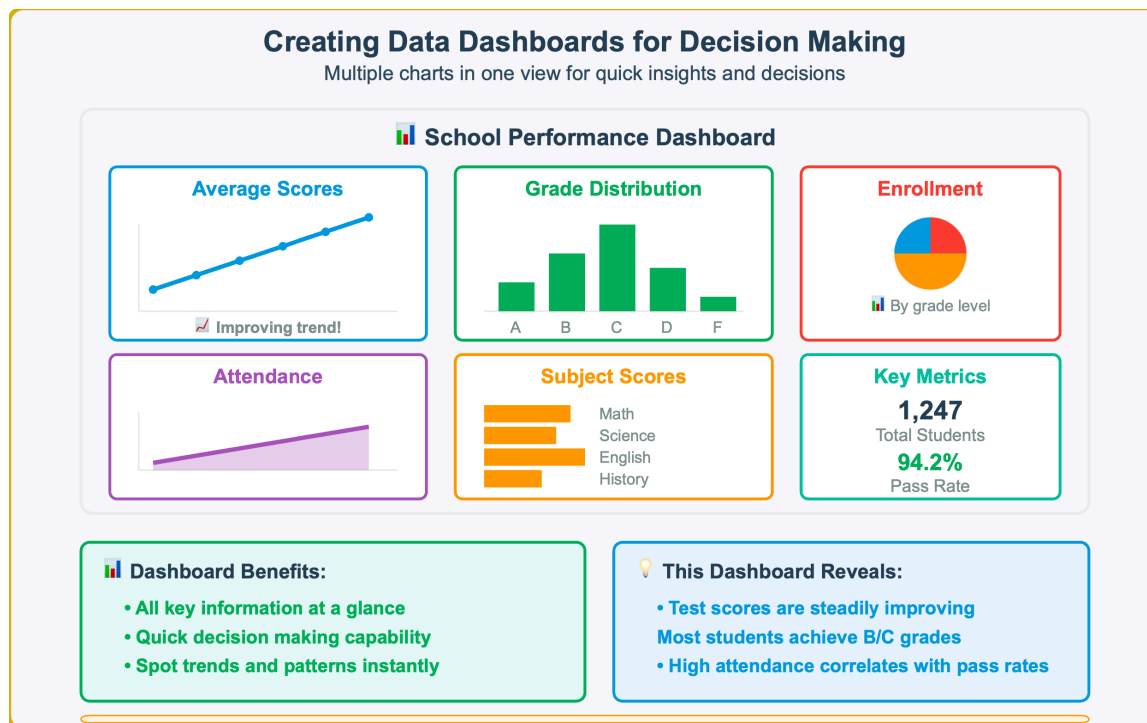


Figure 13.13 — Creating Data Dashboards for Decision Making. A school performance dashboard demonstrates how combining six different chart types (line, bar, pie, area, horizontal bar, and key metrics) provides comprehensive insights at a glance for quick decision making.

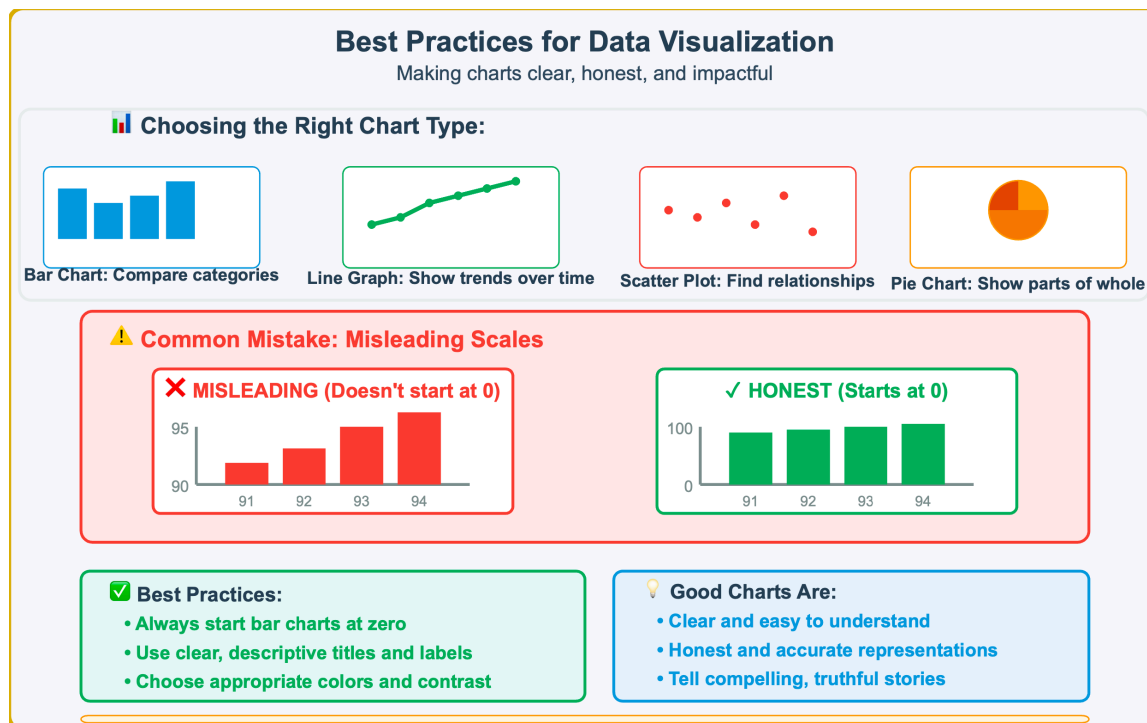


Figure 13.14 — Best Practices for Data Visualization. This guide demonstrates proper chart selection, honest scaling practices, and the importance of starting bar charts at zero to avoid misleading representations. Good visualizations are clear, accurate, and tell compelling stories without distorting the data.

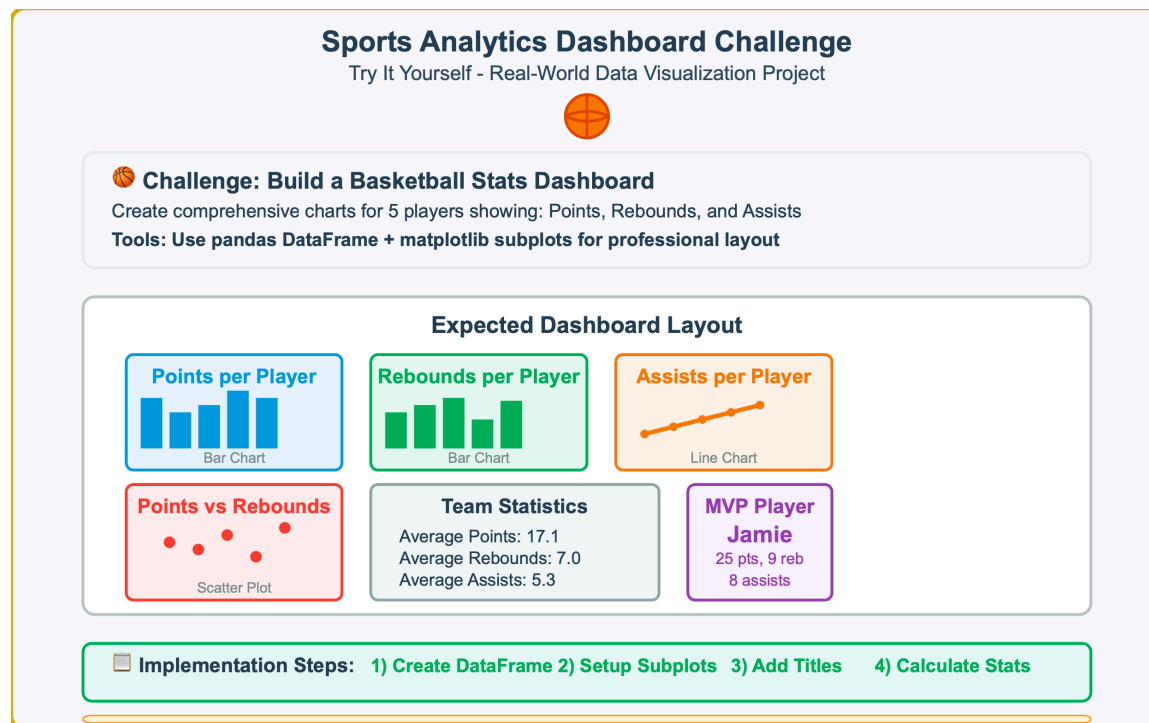


Figure 13.15 — Sports Analytics Dashboard Challenge. This hands—on project challenges students to create a basketball statistics dashboard using pandas DataFrame and matplotlib subplots, combining points, rebounds, assists charts with team statistics for comprehensive player analysis.

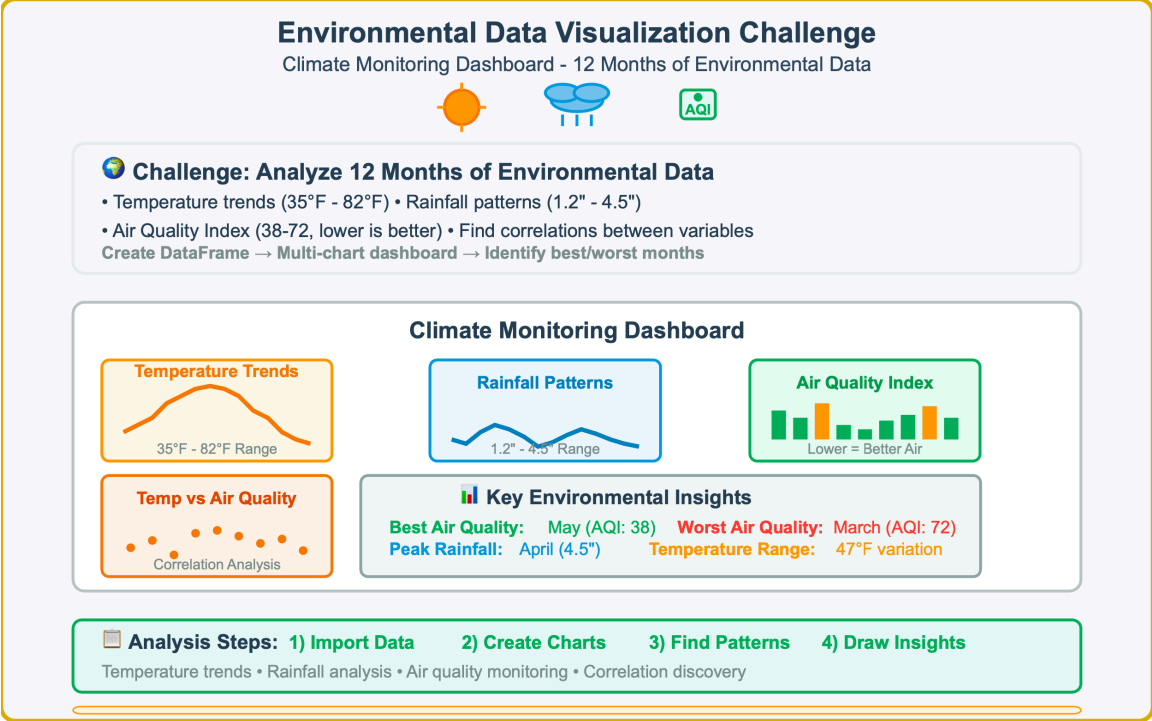


Figure 13.16 — Environmental Data Visualization Challenge. This climate monitoring project analyzes 12 months of environmental data including temperature trends, rainfall patterns, and air quality index measurements to create a comprehensive dashboard revealing seasonal patterns and correlations.

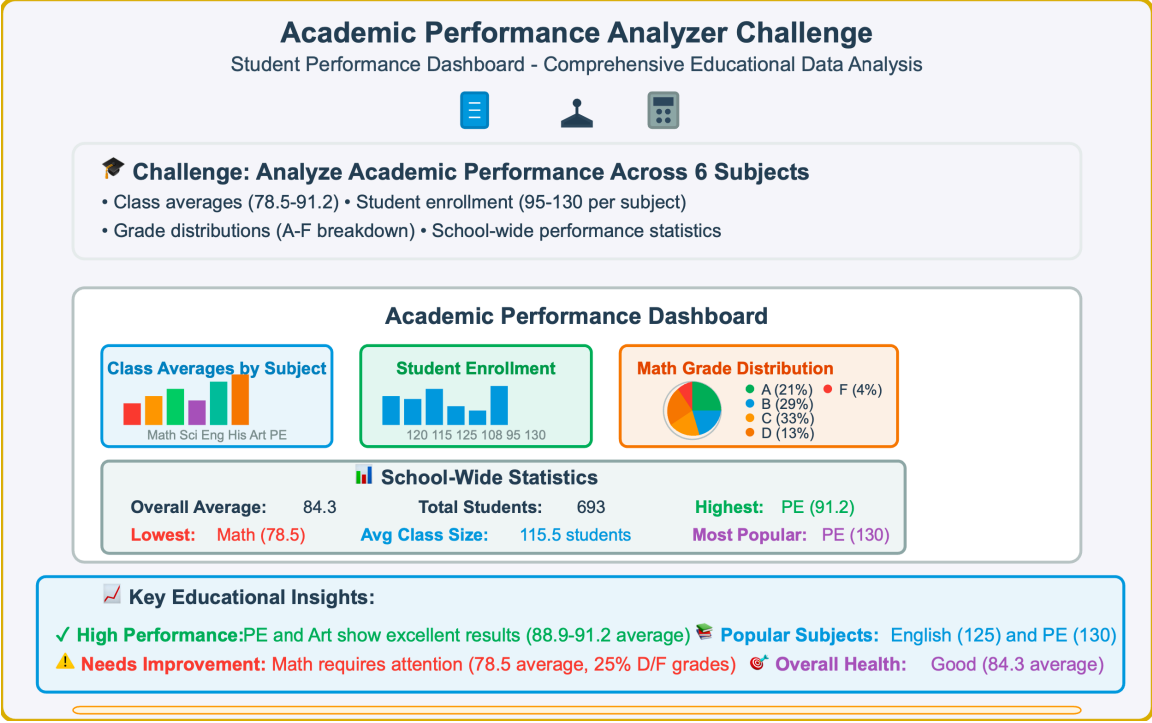


Figure 13.17 — Academic Performance Analyzer Challenge. This educational data analysis project examines student performance across six subjects, analyzing class averages, enrollment numbers, grade distributions, and school—wide statistics to identify trends and areas for improvement.